CS 3244

Machine Learning

Week 11 Post-Class
GMM and Expectation Maximation

11 post

Kan Min-Yen
Quoc Trung Luong

Background Photo credits: Rafiq Mirza, Luan Ahn, Rosmarie Voegtli @ Flickr

Recap and Forecast

Decision Tree Learning (DTL)

- Expressive: can overfit to achieve $J_{train} = 0$
- Need pruning to combat variance

Extensions: Diversity is good

Bootstrap & Bagging: Sample with replacement to generate new $\widetilde{\mathcal{D}}_t$.

- With each, train a h_t, apply uniform voting.
- Variance lowered through averaging.

Boosting: Make a strong classifier from simple weak classifiers.

- Reweight $\widetilde{\mathcal{D}}_t$ to account for easy / hard examples.
- Lower bias via simple hypotheses.

Random Forest: Combine *Bagging* with DTL and introduce feature projection.

- Each h_t tuned to some subset of the features.

Learning Outcomes for this week:

- Have a basic understanding of unsupervised learning;
- Understand different unsupervised learning algorithms for:
  - Dimensionality reduction: For aiding visualization, exploratory analysis;
  - Clustering: Finding latent patterns.
- Implement and understand k means clustering;
- Understand the Expectation Maximization (EM) meta-algorithm;
- Apply unsupervised learning on real-world data.

Gaussian Mixture Model

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Supervised classification?

Assume you build a solution to a homework assignment!

I tell you it will be a supervised problem, so you build a supervised classifier.

You study the problem and decide which model you are going to use and then you are told:

"Sorry, we don't have the annotations $\boldsymbol{y}$ ready. I can only give you the unlabeled data $\mathbf{X}$."

"Whaaat?"

The Expectation-Maximization family

Relax. Can!

Just treat the labels as hidden variables, to be learned alongside the parameters of the model.

Leads to an alternating, iterative solution, similar in spirit to K means: hold one variable fixed to estimate the other(s).

Gaussian Mixture Models (GMM)

GMMs can model data that comes from one of several known groups, where each group can be modeled by a Gaussian distribution.

Example:

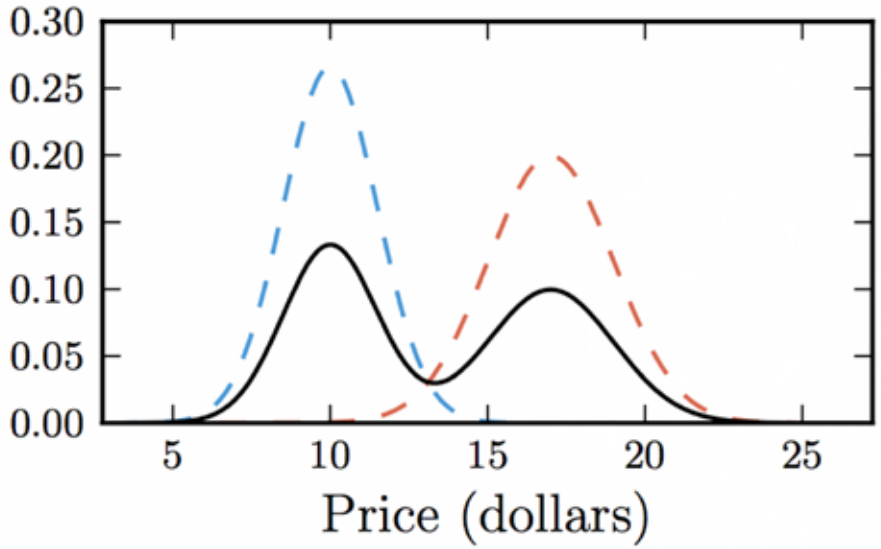
The price of a paperback book is normally distributed with mean price $10.00 and standard deviation $1.00.

Hardbacks are also normally distributed with mean $17 and variance $1.50

<u>Quick Question</u>: Is the price of a randomly chosen book normally distributed?

This slide's example is from MIT

No, it's bimodal.



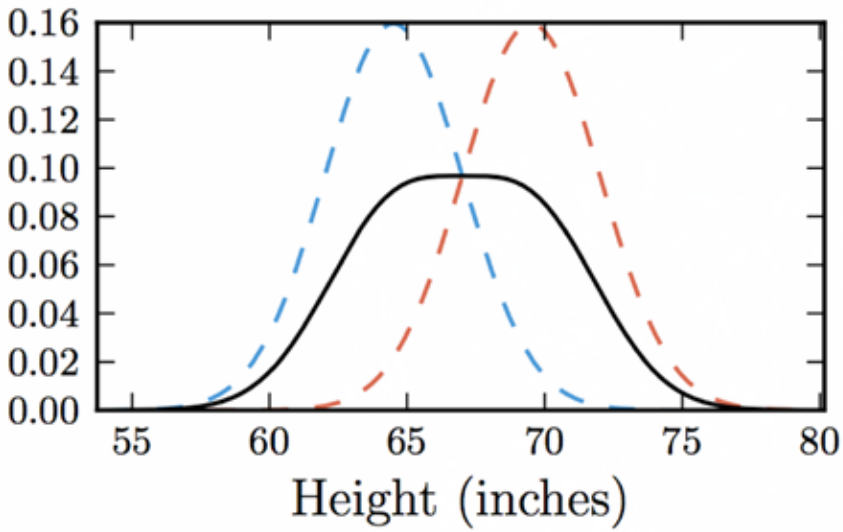
Quick Question 2:

Say men's heights are normally distributed with mean 177cm, and women's heights are also normally distributed at 164cm. Both have standard deviations of about 6cm.

Is the height of a random person normally distributed?

This slide's example is from MIT

No, although the overall distribution is highest at the center.



Both examples are models that are mixtures of Gaussians.

Can we recover the component Gaussian distributions from the final distribution?

That's exactly what the EM framework does.

This slide's example is from MIT

MLE for Mixture Distributions

When we proceed to calculate the maximum likelihood estimate for a mixture, the presence of the sum of the distributions prevents a "neat" factorization using the log function.

Parameters of the model

$$P(\mathcal{D}|\theta) = \prod_{j=1}^{m} P(\mathbf{x}^{(j)}|\theta)$$

$$P(\mathcal{D}|\theta) = \prod_{j=1}^{m} \left(\sum_{k=1}^{K} P(\mathbf{x}^{(j)}, y^{(j)} = k|\theta) \right)$$

Mixture of K distributions

$$log(P(\mathcal{D}|\theta)) = \sum_{j=1}^{m} log \left(\sum_{k=1}^{K} P(\mathbf{x}^{(j)}, y^{(j)} = k|\theta) \right)$$

The inner sum makes it hard to find MLE

A different tack is needed to estimate the parameters.

Missing Data

We can think of clustering as a problem of estimating missing data.

The missing data are the cluster labels.

Clustering is just one instance of missing data problems.

Several other learning problems can be also formulated as missing data problems.

K class GMM with labels

Let's take a step back and think about how we would do this if we were provisioned with labels.

We can write the generative story for this model as:

for each of the j examples 1 to m:

choose a label $y^{(j)} \sim Mult(\pi)$.

choose example $\mathbf{x}^{(j)} \sim Normal\left(\mu_{y^{(j)}}, \sigma^2_{y^{(j)}}\right)$.

K class GMM with labels

This translates into a likelihood as follows:

$$P(\mathcal{D}|\theta) = \prod_j P(\mathbf{x}^{(j)}, y^{(j)}|\pi, \mu_1, \dots, \mu_K, \sigma_1^2, \dots, \sigma_K^2)$$

$$= \prod_j Mult(y^{(j)}|\pi) Normal(x^{(j)}|\mu_{y^{(j)}}, \sigma_{y^{(j)}}^2)$$

$$= \overbrace{\prod_j \underbrace{\pi_{y^{(j)}}}_{\text{choose label}} \underbrace{\left[2\pi\sigma_{y^{(j)}}^2\right]^{-\frac{1}{2}} \exp\left[-\frac{1}{2\sigma_{y^{(j)}}^2}\left|\left|x^{(j)} - \mu_{y^{(j)}}\right|\right|^2\right]}_{\text{choose feature value}}}^{\text{for each example}}$$

K class GMM with labels

Then we can obtain closed-form solutions for the maximum likelihood estimates (MLE) of the parameters by taking a log and then gradients of the log likelihood:

$$\pi_k = \text{fraction of examples in class } k$$
$$= \frac{m_k}{m}$$
$$\mu_k = \text{mean of examples in class } k$$
$$= \frac{1}{m_k}\sum_{j=1}^{m_k} \mathbf{x}^{(j)}$$
$$\sigma_k^2 = \text{variance of examples in class } k$$
$$= \frac{1}{m_k}\sum_{j=1}^{m_k} \left\|\mathbf{x}^{(j)} - \mu_k\right\|^2$$

Ok, so what?

We don't have the labels!

K class GMM without labels

Similarly, we want to maximize the marginal log-likelihood of $\mathcal{D}$

$$log(P(\mathcal{D}|\theta)) = \sum_{j=1}^{m} log\left(P(\mathbf{x}^{(j)}|\theta)\right)$$

Since our data only have x

Where $\theta = (\pi, \mu_1, \dots, \mu_K, \sigma_1^2, \dots, \sigma_K^2)$

Example $\mathbf{x}^{(j)}$ can be generated by any distribution in mixture Think of latent variable $y^{(j)}$ as a random variable:

$$y^{(j)}|\mathbf{x}^{(j)} \sim Mult(z_1^{(j)}, z_2^{(j)}, \dots, z_K^{(j)})$$

where $z_k^{(j)}$ estimates the expected label of $\mathbf{x}^{(j)}$ in cluster k

$\rightarrow z_k^{(j)}$ estimates $P\left(y^{(j)} = k \middle| \mathbf{x}^{(j)}, \theta^*\right)$

This $\boldsymbol{z}$ only related to target θ^*, update $\boldsymbol{z}$ doesn't change the value of marginal log-likelihood but it helps with interpretation.

Maximize the Likelihood

<u>Problem</u>: The "unfriendly" form of the marginal log-likelihood makes it hard to explicitly derive MLE.

$$log(P(\mathcal{D}|\theta)) = \sum_{j=1}^{m} log\left(\sum_{k=1}^{K} P(\mathbf{x}^{(j)}, y^{(j)} = k|\theta)\right)$$

<u>One trick</u>: Find a "friendly" lower bound and maximize it instead.

$$log(P(\mathcal{D}|\theta)) = \sum_{j=1}^{m} log\left[\sum_{k=1}^{K} z_k^{(j)} \frac{P(\mathbf{x}^{(j)}, y^{(j)} = k|\theta)}{z_k^{(j)}}\right]$$

Jensen's inequality

$$\geq \sum_{j=1}^{m}\sum_{k=1}^{K} z_k^{(j)} \, log\left[\frac{P(\mathbf{x}^{(j)}, y^{(j)} = k|\theta)}{z_k^{(j)}}\right]$$

Easier to take derivative and find MLE

Aside: Jensen's inequality

Given a **concave** function f with number $\{x_1, x_2, \dots x_n\}$ in its domain and positive weight that sum to unity $\{\lambda_1, \lambda_2, \dots \lambda_n\}$, Jensen's inequality states:

$$f\left(\sum_i \lambda_i x_i\right) \geq \sum_i \lambda_i f(x_i)$$

The equality occurs when $x_1 = x_2 = \cdots = x_n$ or f is linear.

- In our problem, the log function is **concave**, with positive weights that sum to unity: $\left\{z_1^{(j)}, z_2^{(j)}, \dots z_K^{(j)}\right\}$
- The bound is tight when:

$$z_k^{(j)} = P\left(y^{(j)} = k \middle| \mathbf{x}^{(j)}, \theta\right)$$

Lower bound LB

$$LB = \sum_{j=1}^{m} \sum_{k=1}^{K} z_k^{(j)} log \left[\frac{P\left(\mathbf{x}^{(j)}, y^{(j)} = k \middle| \theta\right)}{z_k^{(j)}} \right]$$

As we can always make the inequality occur by setting:

$$z_k^{(j)} \leftarrow P\left(y^{(j)} = k \middle| \mathbf{x}^{(j)}, \theta\right)$$

The objective of maximizing $P(\mathcal{D}|\theta)$

$\equiv$ maximizing LB

Maximize Lower Bound

Iteratively until convergence:

E. Step: Update latent distribution to make the bound tight:

$$z_k^{(j)} \leftarrow P(y^{(j)}|\mathbf{x}^{(j)}, \theta)$$

→ The marginal log-likelihood remains

M. Step: Update model parameters to remain/increase LB:

$$\theta \leftarrow \arg\max_{\theta}(LB)$$

→ The marginal log-likelihood must also remain/increase

- Both steps either remain or increase the marginal log-likelihood.
- As $\theta \rightarrow \theta^*$, $z_k^{(j)}$ estimates $P(y^{(j)}|\mathbf{x}^{(j)}, \theta^*)$ better

Simplified View

$$LB = \underbrace{\sum_{j=1}^{m}\sum_{k=1}^{K} z_k^{(j)} log\left[P\left(\mathbf{x}^{(j)}, y^{(j)} = k \middle| \theta\right)\right]}_{(*)} - H(z)$$

$(*)$ is the expected value of the log-likelihood, where labels are sampled from an estimate of their actual distribution:

$$y^{(j)}|\mathbf{x}^{(j)} \sim Mult(z_1^{(j)}, z_2^{(j)}, \dots, z_K^{(j)})$$

$$LB = \mathbb{E}_{y^{(1)},\dots,y^{(m)}}\left[\sum_{j=1}^{m} log\left[P(\mathbf{x}^{(j)}, y^{(j)}|\theta)\right]\right] - H(z)$$

Simplified View

While the mathematic for LB, Jensen's inequality is useful to know, we don't need it to actually perform model estimation.

All we need is the expected value of the log-likelihood:

$$(*) = \sum_{j=1}^{m} \sum_{k=1}^{K} z_k^{(j)} log\left[P\left(\mathbf{x}^{(j)}, y^{(j)} = k \middle| \theta\right)\right]$$

Iterative until convergence:

- Expectation Step: update latent distribution according to current model parameters θ:

$$z_k^{(j)} \leftarrow P(y^{(j)} | \mathbf{x}^{(j)}, \theta)$$

- Maximization Step: update θ to maximize the expected value of the log-likelihood, treating latent variables as observed:

$$\theta \leftarrow \arg\max_{\theta}((*))$$

Hence the name Expectation Maximization

E–Step for the GMM

$$z_k^{(j)} = P\left(y^{(j)} = k \middle| \mathbf{x}^{(j)}, \theta\right)$$

$$= \frac{P\left(y^{(j)} = k, \mathbf{x}^{(j)} | \theta\right)}{P(\mathbf{x}^{(j)} | \theta)}$$

$$= \frac{1}{Z^{(j)}} Mult(k|\pi) Normal(\mathbf{x}^{(j)} | \mu_k, \sigma_k^2)$$

where $Z^{(j)}$ is a normalizer to ensure $\mathbf{z}^{(j)}$ sums to unity.

M–Step from the GMM

$$(*) = \sum_{j=1}^{m} \sum_{k=1}^{K} z_k^{(j)} log\left[Mult(k|\pi) Normal(\mathbf{x}^{(j)}|\mu_k, \sigma_k^2)\right]$$

Take derivative w.r.t each parameters to find MLEs:

$$\mu_k = \frac{\sum_j z_k^{(j)} \mathbf{x}^{(j)}}{\sum_j z_k^{(j)}}$$

$$\sigma_k^2 = \frac{\sum_j z_k^{(j)} \left\|\mathbf{x}^{(j)} - \mu_k\right\|^2}{\sum_j z_k^{(j)}}$$

$$\pi_k = \frac{1}{m} \sum_j z_k^{(j)}$$

K class GMM without labels

In summary, we perform 2 steps iteratively:

1. E. Step:

$$z_k^{(j)} = \text{probabilistic label of } \mathbf{x}^{(j)} \text{ in class } k$$
$$= \frac{1}{Z^{(j)}} Mult(k|\pi) Normal(\mathbf{x}^{(j)}|\mu_k, \sigma_k^2)$$

2. M. Step:

$$\mu_k = \text{mean of examples in class } k$$
$$= \frac{\sum_j z_k^{(j)} \mathbf{x}^{(j)}}{\sum_j z_k^{(j)}}$$
$$\sigma_k^2 = \text{variance of examples in class } k$$
$$= \frac{\sum_j z_k^{(j)} \|\mathbf{x}^{(j)} - \mu_k\|^2}{\sum_j z_k^{(j)}}$$
$$\pi_k = \text{fraction of examples in class } k$$
$$= \frac{1}{m} \sum_j z_k^{(j)}$$

EM for GMMs

for $k = 1$ to K do: # initialize randomly

$\mu_k \leftarrow$ some random location, $\sigma_k^2 \leftarrow 1, \pi_k \leftarrow 1/K$

repeat until converged:

for $j = 1$ to m do:

for $k = 1$ to K do: # E. Step

$$z_k^{(j)} \leftarrow \pi_k \left[2\pi\sigma_k^2\right]^{-\frac{1}{2}} \exp\left[-\frac{1}{2\sigma_k^2}\left|\left|\mathbf{x}^{(j)} - \mu_k\right|\right|^2\right]$$

$$z^{(j)} \leftarrow \frac{1}{\sum_k z_k^{(j)}} z^{(j)}$$

for $k = 1$ to K do: # M. Step

$$\pi_k \leftarrow \frac{1}{m}\sum_j z_k^{(j)}$$

$$\mu_k \leftarrow \frac{\sum_j z_k^{(j)}\mathbf{x}^{(j)}}{\sum_j z_k^{(j)}}, \sigma_k^2 \leftarrow \frac{\sum_j z_k^{(j)}\left|\left|\mathbf{x}^{(j)} - \mu_k\right|\right|^2}{\sum_j z_k^{(j)}}$$

return z

Generalize to other distributions

Previously, we deal with missing labels for discrete random variable in the multinomial distribution. We can apply the same for missing variables for any distribution.

Our objective is still to maximize the marginal likelihood of the data:

$$log(P(\mathcal{D})) = \sum_{j=1}^{m} log\left(\sum_{y^{(j)}} P(\mathbf{x}^{(j)}, y^{(j)} | \theta)\right)$$

Optimization is still the same:

$$LB = \underbrace{\sum_{j=1}^{m} \sum_{y^{(j)}} Q_j(y^{(j)})\, log(P(\mathbf{x}^{(j)}, y^{(j)} | \theta)) - H(Q)}_{(*)}$$

* NB: for continuous random variables, replace sum with integral and P with density function

- Update latent distribution Q according to current parameter θ:
$$Q_j(y^{(j)}) \leftarrow P(y^{(j)} | \mathbf{x}^{(j)}, \theta)$$
- Maximize the expected value of the log likelihood: $\theta \leftarrow \arg\max_{\theta}((*))$

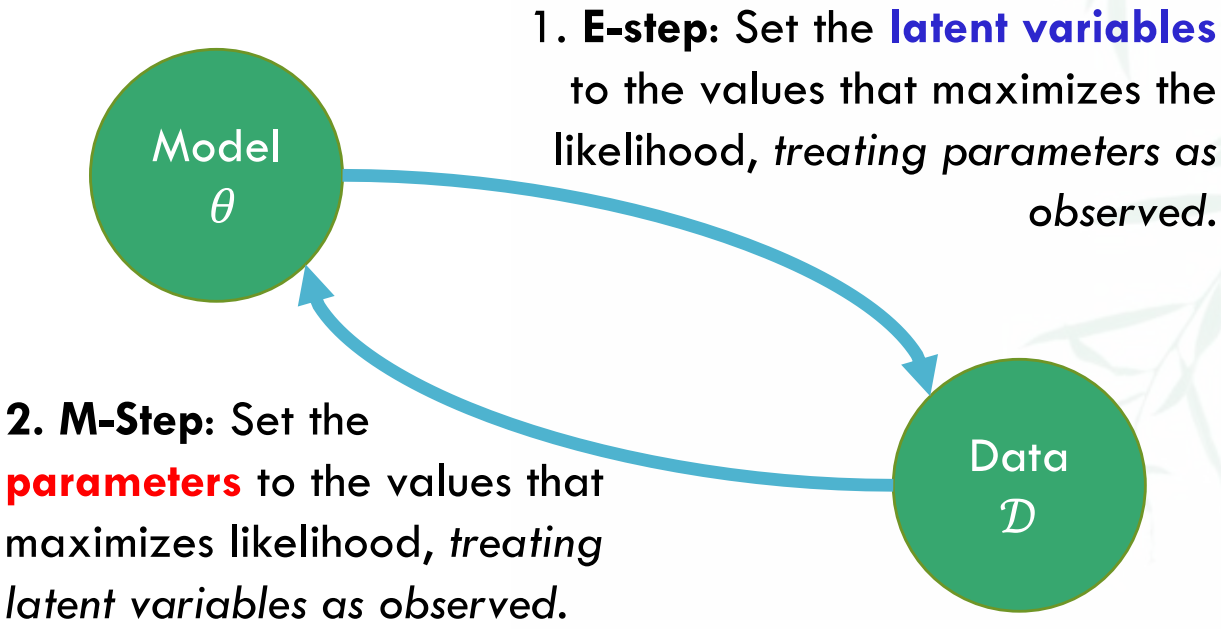
Expectation Maximization

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

The general EM framework

A model θ of the data generating process gives rise to $\mathcal{D}$. Model estimation from data done through some form of likelihood estimation.



But that's K means!

Yes, you're right.

Instead of maximizing likelihood, K means minimizes sum squared error objective which is also a valid heuristic.

K means assigns point to exactly one cluster, the $\mathcal{L}^2$ norm is minimized only over this domain.

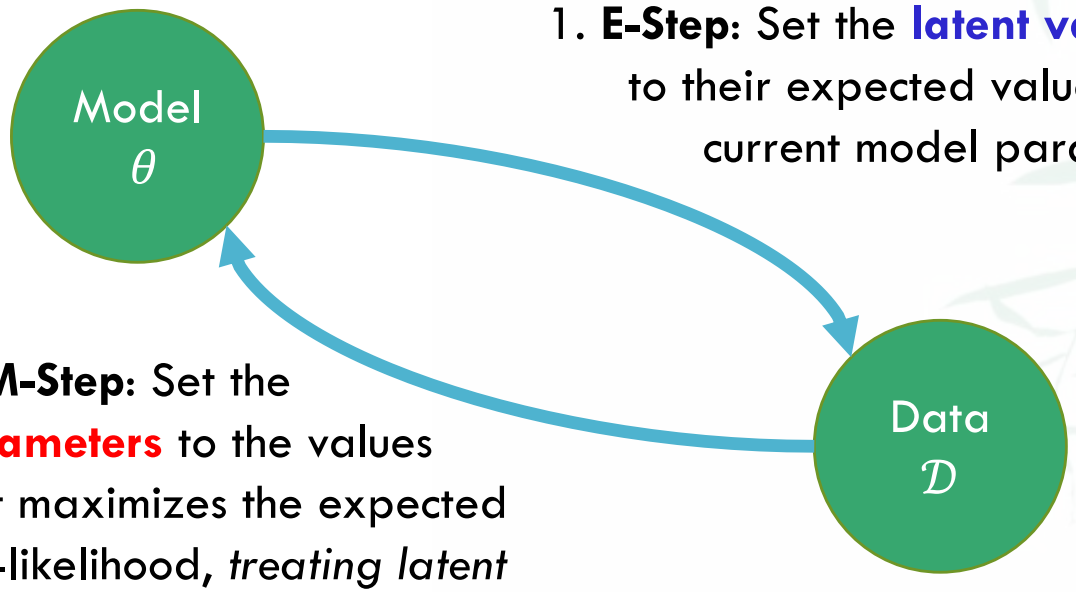
→ We can think of K means as a "hard" form of EM

GMM uses "soft" assignment, the likelihood is maximized over the domain that a point is assigned to many clusters.

- "Hard" assignment in GMM is when we impose a constrain that $z_k^{(j)}$ is either 0 or 1, but then the likelihood is hard to maximize.

The popular EM framework

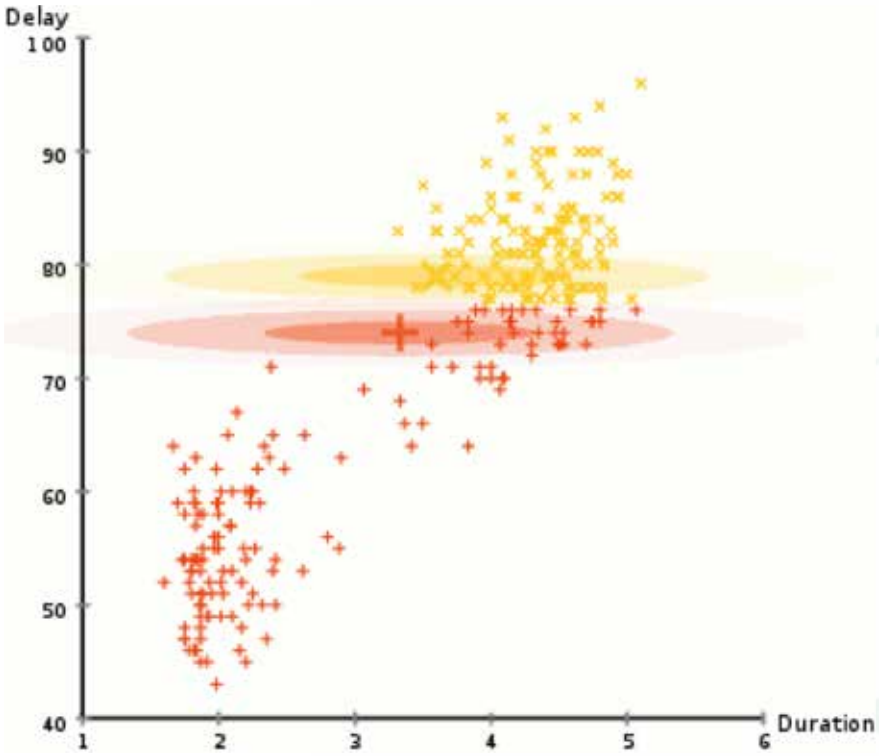
This version is more specific, it is exactly the EM used in GMM. The likelihood is maximized by maximizing the expected log-likelihood. Its correctness can be proven.



1. **E-Step:** Set the **latent variables** to their expected values given current model parameters.

2. M-Step: Set the **parameters** to the values that maximizes the expected log-likelihood, *treating latent variables as observed.*

EM Example



EM clustering of Old Faithful eruption data. The random initial model (which, due to the different scales of the axes, appears to be two very flat and wide spheres) is fit to the observed data. In the first iterations, the model changes substantially, but then converges to the two modes of the geyser. Visualized using ELKI.

By Chire - Own work, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=20494862

Properties of EM

EM is a local optimization function, but often the likelihood function is not convex.

We'll need to run EM several times with different random initializations and hope for the best.

Pick the parameters that yield the highest likelihood.

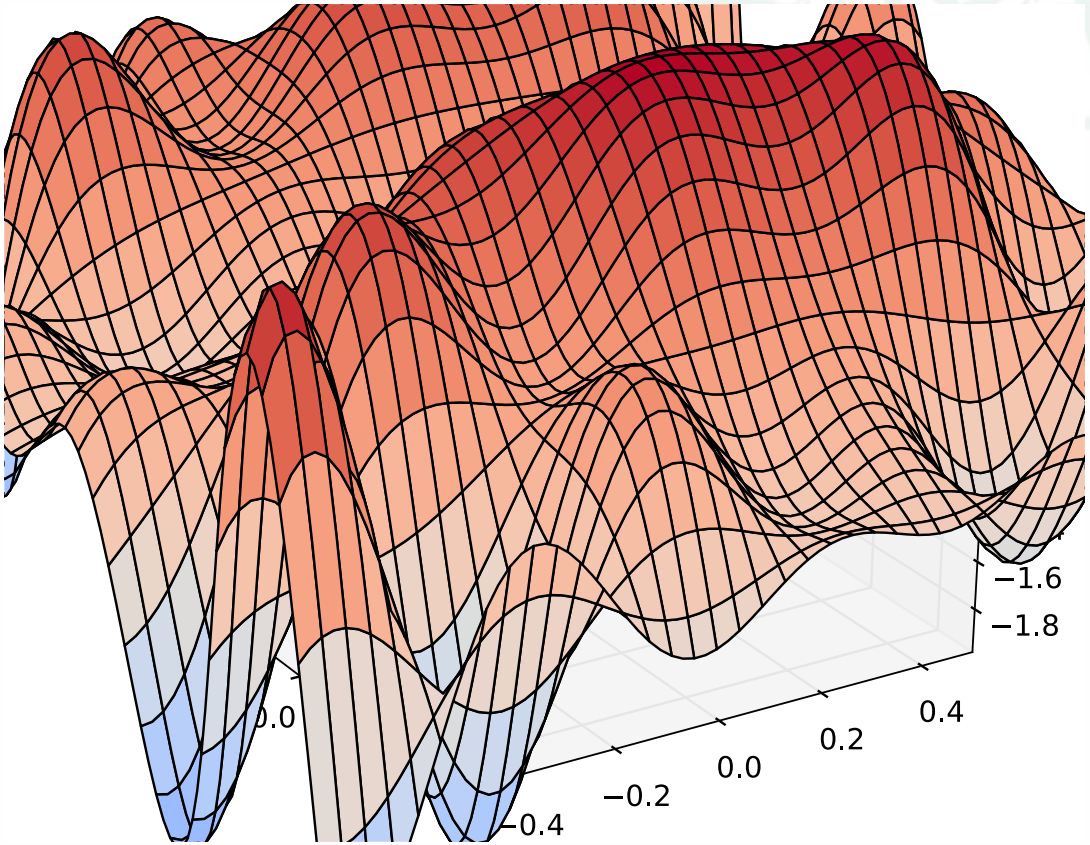


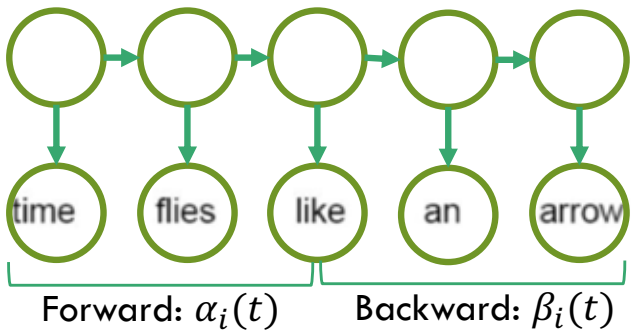
Photo credits: Matt Gormley @ CMU,

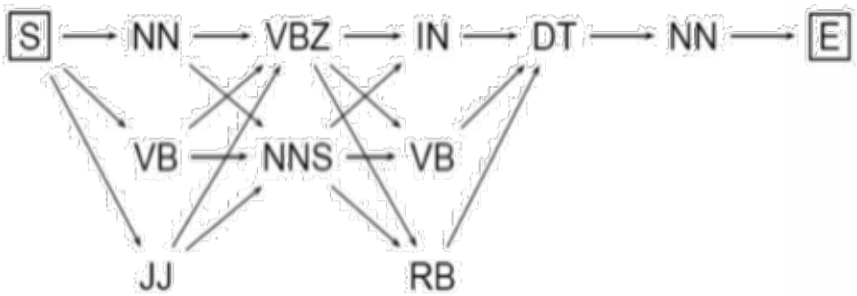
EM by other names

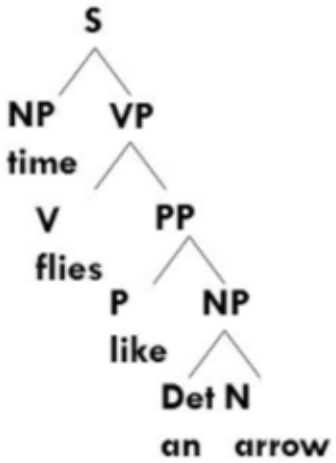
(Examples from natural language processing)

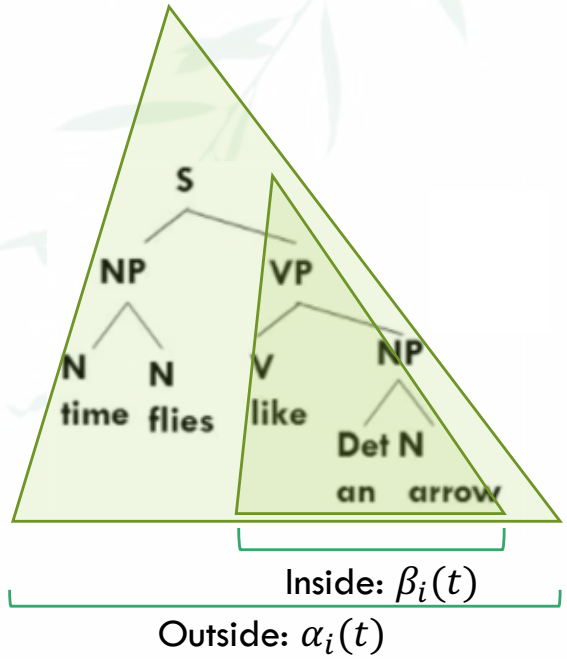
Forward Backward: Use EM for estimating latent labels on sequential models (HMM: Hidden Markov Model)

Inside Outside: Use EM for tree-like structures (like PCFGs: probabilistic context-free grammars)









Variants of EM

Generalized EM: Replace the M-Step by a single gradient-step that improves the likelihood

Monte Carlo EM: Approximate the E-Step by sampling

Sparse EM: Keep an "active list" of points (updated occasionally) from which we estimate the expected counts in the E-Step

Incremental EM / Stepwise EM: If standard EM is described as a *batch* algorithm, these are the *online* equivalent.

Which of these variants are good for what scenarios?

Unsupervised Learning: Summary

CS3244 Machine Learning

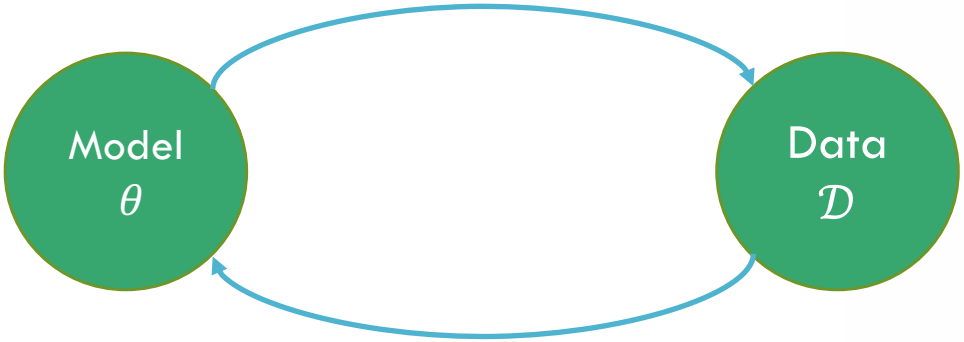
NUS National University of Singapore
Department of Computer Science
School of Computing

Summary

Clustering

Expectation Maximization

E: Assign data to model



M: Likelihood of the model

K means as as simplified form of the EM algorithm, where cluster assignments are hard, and using an $\mathcal{L}^2$ norm.

Dimensionality Reduction

Principal Component Analysis

Find a low rank approximation of the original data.

Use orthonormal bases with the most variance to determine principle components.

Use SVD to achieve PCA for practicality.

Can we put together both practices – iterative estimation & dimensionality reduction? Yes! One way is **t-SNE** dimensionality reduction, popularized in Tensorflow and sklearn.